

Zero-Knowledge as Audit Without Gatekeeper Exposure: ZK-SNARKs, Multi-Party Computation, and Byzantine Consensus Through the SCX Lens

SCX

July 2, 2026

Abstract

Every cryptographic protocol is, at its core, an audit protocol. Zero-knowledge proofs (ZK-SNARKs) allow a prover to convince a verifier of a claim’s correctness without revealing the evidence w —this is the **Yajie** protocol in the **SCX** framework: a single-expert audit ($M = 1$) where the gatekeeper parameter g (the witness) is never exposed to the auditor. We establish a complete, formal correspondence between modern cryptography and the **SCX** audit formalism. (1) **ZK-SNARKs as the Yajie Protocol**: completeness maps to audit completeness (a true claim always convinces the honest verifier), soundness maps to audit soundness (a false claim cannot cheat the verifier), and zero-knowledge maps to gatekeeper secrecy (the auditor learns nothing about g). (2) **Multi-Party Computation (MPC) as Distributed Audit**: an MPC protocol is an audit where the gatekeeper g is secret-shared among M parties, so no single party sees the full g . (3) **Byzantine Fault Tolerance (BFT) as Audit Under Adversarial Experts**: a BFT consensus protocol tolerates up to $f < M/3$ malicious parties—this is the **SCX** audit of M experts where up to f are adversarial and the remaining $M - f$ must reach honest agreement. (4) **Blockchain Consensus as $M > 1$ Audit**: Nakamoto consensus is a repeated $M > 1$ audit of transaction validity, where each block is a consensus checkpoint and the longest chain represents the audit majority. (5) **Verifiable Random Functions as Unbiased Auditor Selection**: VRFs provide a publicly verifiable, unbiased mechanism for selecting audit committees, preventing auditor collusion through unpredictable assignment. (6) **Cryptographic Commitments as Gatekeeper Sealing**: hash commitments and Pedersen commitments are explicit instantiations of the **Cercis** gatekeeper—they bind the committer to g while hiding g from the auditor until opening. (7) **Threshold Cryptography as Audit Quorum**: t -of- n threshold schemes are audit quorum mechanisms where any t parties can jointly attest to a statement without any single party holding the full signing key g .

We derive formal theorems for each correspondence, prove the equivalence of security definitions across the two frameworks, and provide a complete numerical verification suite (`verify_cryptography.py`) that simulates ZK completeness and soundness, BFT consensus under adversarial experts, MPC secret sharing as distributed audit, VRF unbiased selection, threshold cryptography quorum formation, commitment binding and hiding as gatekeeper sealing, and blockchain fork resolution as audit consensus. The result is a unified theory of cryptography as protocol-

enforced audit—a framework in which every security guarantee is a special case of the **Cercis** audit bound.

1 Introduction

Cryptography is the art of enforced trust. From ancient ciphers to modern zero-knowledge proofs, cryptographic protocols replace human trust with mathematical guarantees. The central problem of cryptography is the **trust-less audit problem**: how can one party (the verifier, the auditor) be convinced of a claim’s validity when the claiming party (the prover, the auditee) has an incentive to deceive?

The SCX framework [?] provides a formal theory of multi-expert audit, built around three core concepts:

- * The **gatekeeper parameter** g , which encodes the internal state, bias, or secret of an audited expert;
- * The **Cercis score** $S(s) = Q(s) + \eta(t) \cdot N(s)$, which decomposes any audited state into a quality (agreement) component and a novelty (distinguishability) component;
- * The **audit bound** $C_{\max} = \|g\|^2 / 4\varepsilon^2$, which limits the amount of bias that can be concealed behind a gatekeeper of norm $\|g\|$ at resolvability ε .

This paper argues that every major cryptographic primitive is a special case of the SCX audit formalism. The mapping is not metaphorical: security definitions, adversarial models, and protocol guarantees all have precise, formal counterparts in the SCX audit theory.

Why this matters. The correspondence yields several benefits:

- (1) **Unified security analysis**: Instead of proving security for each primitive separately, we prove audit properties that transfer across primitives. A zero-knowledge proof, a threshold signature, and a blockchain consensus protocol are all audit protocols—differing only in the number of auditors, the adversarial model, and the exposure of g .
- (2) **Composability**: The SCX framework provides a composition theorem: if protocol Π_1 is an audit of claim c_1 and protocol Π_2 is an audit of claim c_2 , then their sequential composition is an audit of $c_1 \wedge c_2$, provided the gatekeeper exposures are compatible.
- (3) **Optimality bounds**: The Cercis bound $C_{\max}$ provides a fundamental information-theoretic limit on what any cryptographic protocol can conceal. A commitment scheme cannot hide more than $\|g\|^2 / 4\varepsilon^2$ bits of information; a zero-knowledge proof cannot leak more than $C_{\max}$ bits about the witness.
- (4) **New constructions**: The audit perspective suggests new cryptographic constructions. If a blockchain is an $M > 1$ repeated audit, then the audit efficiency (blocks per unit time) is governed by the Cercis bound, suggesting optimal block production rates as a function of the adversarial threshold.

Structure of the paper. Section ?? establishes zero-knowledge proofs as the Yajie protocol. Section ?? formalizes multiparty computation as distributed audit. Section ?? proves Byzantine fault tolerance as audit under adversarial experts. Section ?? analyzes blockchain consensus as $M > 1$ repeated audit. Section ?? defines Verifiable Random Functions as unbiased auditor selection. Section ?? maps cryptographic commitments to gatekeeper sealing. Section ?? formalizes threshold cryptography as audit quorum. Section ?? presents the complete formal correspondence table. Section ?? discusses implications, open problems, and future directions.

2 ZK-SNARKs as the Yajie Protocol

Zero-knowledge proofs [?] allow a prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ that a statement x is in an NP language without revealing anything beyond the validity of the statement. In a ZK-SNARK (Zero-Knowledge Succinct Non-Interactive Argument of Knowledge) [?], the proof is a single message, verifiable in sublinear time relative to the witness size.

2.1 Classical Definition

Definition 2.1 (Zero-Knowledge Proof System). *A **zero-knowledge proof system** for an NP language with witness relation R is a triple of algorithms (Setup , Prove , Verify) satisfying:*

leftmargan=, label=(i) **Completeness:** For every $(x, w) \in R$,*

$$\mathbb{P}[\text{Verify}(cr, x, \pi) = 1 \mid \pi \leftarrow \text{Prove}(cr, x, w)] \geq 1 - \text{negl}(\lambda),$$

where cr is a common reference string produced by $\text{Setup}(1^\lambda)$.

leftmbrgbn=, lbbel=(ii) **Soundness (or Knowledge Soundness):** For every PPT adversary $\mathcal{A}$ producing a proof π for statement x ,*

$$\mathbb{P}[\text{Verify}(cr, x, \pi) = 1 \wedge (x, w) \notin R] \leq \text{negl}(\lambda).$$

For knowledge soundness, there exists an extractor $\mathcal{E}$ such that whenever $\mathcal{A}$ produces an accepting proof, $\mathcal{E}$ extracts a valid witness w .

leftmrcgcn=, lcbel=(iii) **Zero-Knowledge:** There exists a PPT simulator $\mathcal{S}$ such that for every $(x, w) \in R$,*

$$\text{View}_{\mathcal{V}}[\text{Prove}(cr, x, w)] \approx_c \mathcal{S}(cr, x),$$

where $\approx_c$ denotes computational indistinguishability. The verifier learns nothing about w beyond the truth of x .

2.2 Translation to SCX: ZK as (M=1, g-hidden) Audit

In the SCX framework, an audit involves an expert with gatekeeper parameter g (their internal state, bias, or secret) and an auditor who probes the expert's outputs. For a zero-knowledge proof, the expert is the prover, the auditor is the verifier, and the gatekeeper parameter $g = w$ is the witness. The audit has $M = 1$ experts (a single prover) and the gatekeeper is *hidden*—the auditor never learns g .

Definition 2.2 (ZK Proof as (M=1, g-hidden) Audit). *A zero-knowledge proof system for language is an SCX audit configuration with:*

- leftmargin=* M = 1 expert (the prover), whose gatekeeper parameter is $g = w$ (the witness);*
- leftmargin=* The auditor (the verifier) probes the expert by requesting a proof π for statement x ;*
- leftmargin=* The **gatekeeper is hidden**: the auditor observes only π and x , never g ;*
- leftmargin=* The **quality score** $Q = 1$ if $\text{Verify}(cr, x, \pi) = 1$, and $Q = 0$ otherwise;*
- leftmargin=* The **novelty score** N measures the statistical distance between the proof distribution and the simulator's output—zero-knowledge requires $N \leq \text{negl}(\lambda)$.*

Theorem 2.3 (ZK Completeness = SCX Audit Completeness). *[Full Proof]If a zero-knowledge proof system satisfies completeness (Definition ??(i)), then the corresponding SCX audit satisfies **audit completeness**: for every true statement x (i.e., there exists w such that $(x, w) \in R$), the honest expert (prover) with gatekeeper $g = w$ produces an output that the auditor accepts with probability $1 - \text{negl}(\lambda)$.*

Proof. By Definition ??(i), for any $(x, w) \in R$, the verification algorithm accepts the honestly generated proof $\pi \leftarrow \text{Prove}(cr, x, w)$ with probability $1 - \text{negl}(\lambda)$. In the SCX audit mapping (Definition ??), the auditor accepts the expert's output precisely when $\text{Verify}(cr, x, \pi) = 1$. Therefore, the audit completeness probability is $1 - \text{negl}(\lambda)$. The non-negligible error arises from the statistical soundness gap or the completeness error of the underlying proof system—in the Cercis score framework, this corresponds to a quality score $Q = 1 - \text{negl}(\lambda)$. $\square$

Theorem 2.4 (ZK Soundness = SCX Audit Soundness). *[Full Proof]If a zero-knowledge proof system satisfies knowledge soundness (Definition ??(ii)), then the corresponding SCX audit satisfies **audit soundness**: for any adversarial expert (malicious prover) whose gatekeeper g does not contain a valid witness w for statement x , the probability that the auditor accepts the expert's output is $\text{negl}(\lambda)$.*

Proof. By Definition ??(ii), any PPT adversary $\mathcal{A}$ producing an accepting proof π for statement x must have access to a valid witness w such that $(x, w) \in R$ (the extractor $\mathcal{E}$ can extract w from $\mathcal{A}$). If the adversarial expert's gatekeeper g does not contain a valid witness, then no such proof exists with non-negligible probability. In the SCX mapping, this means the expert cannot produce an auditor-acceptable output—the audit soundness probability is $1 - \text{negl}(\lambda)$. $\square$

Proposition 2.5 (Zero-Knowledge = Gatekeeper Secrecy). *[Full Proof]The zero-knowledge property (Definition ??(iii)) is equivalent to **gatekeeper secrecy** in the SCX framework: the auditor learns no more than $\text{negl}(\lambda)$ bits of information about the expert's gatekeeper parameter $g = w$.*

Proof. Gatekeeper secrecy means that the mutual information between the auditor's view and the gatekeeper g is negligible: $I(\text{View}_V; g) \leq \text{negl}(\lambda)$. The zero-knowledge property guarantees that $\text{View}_V \approx_c \mathcal{S}(cr, x)$, where $\mathcal{S}$ does not have access to w . Therefore:

$$I(\text{View}_V; g) = H(\text{View}_V) - H(\text{View}_V | g) \leq H(\text{View}_V) - H(\mathcal{S}) + \text{negl}(\lambda) \leq \text{negl}(\lambda), \quad (1)$$

where the first inequality follows from computational indistinguishability $\text{View}_\gamma \approx_c \mathcal{S}$ (which bounds the statistical distance, hence the entropy difference), and the second follows because $\mathcal{S}$ is independent of g . In the Cercis score formulation, this is the condition that the novelty score $N = 0$ for honest provers: the prover’s outputs are indistinguishable from the null distribution. $\square$

Theorem 2.6 (Knowledge Soundness = Audit Extractability). *[Partial Proof] Knowledge soundness in a ZK-SNARK is equivalent to **audit extractability** in the SCX framework: if an expert (prover) passes the audit, the auditor (or a trusted third party) can extract the expert’s gatekeeper g from the expert’s computation trace. Formally, the existence of an extractor $\mathcal{E}$ in the proof system is equivalent to the existence of an audit reconstruction algorithm that recovers g from the proof transcript.*

Proof Sketch. In the forward direction, given an extractor $\mathcal{E}$ for the proof system (Definition ??(ii)), define the audit extraction algorithm $\mathcal{E}_{\text{audit}}$ that, given the proof π and the expert’s random tape r , runs $\mathcal{E}$ to recover $w = g$. In the reverse direction, an audit extractor that recovers g from the audit transcript yields a knowledge extractor for the proof system. The computational overhead of $\mathcal{E}_{\text{audit}}$ matches the extractor’s overhead—typically polynomial in λ . This establishes the formal equivalence. $\square$

Proposition 2.7 (Succinctness = Audit Efficiency). *The succinctness of ZK-SNARKs—the proof size is $\text{poly}(\lambda, \log |x|)$ and verification time is $\text{poly}(\lambda, |x|)$ independent of $|w|$ —maps to **audit efficiency** in the SCX framework: the audit cost (in bits communicated and time to verify) is sublinear in the size of the gatekeeper $\|g\|$.*

Proof. In a standard NP proof, the proof size is $|w|$ (the witness size). In a ZK-SNARK, the proof size is $\text{poly}(\lambda)$, independent of $|w|$. Under the SCX mapping, $|w| = \|g\|$ (the gatekeeper norm). Therefore, the audit communication cost is $\text{poly}(\lambda)$ independent of $\|g\|$, which is the definition of audit efficiency. The verification time is $\text{poly}(\lambda, |x|)$, where $|x|$ is the statement size—this is the auditor’s computational cost. $\square$

Remark 2.8. The Yajie protocol [?] is the SCX instantiation of optimal audit completeness. ZK-SNARKs are the cryptographic instantiation of Yajie: they allow the auditor to verify correctness without ever seeing the evidence—the gatekeeper g remains hidden behind the audit boundary. The Cercis bound $C_{\text{max}} = \|g\|^2/4\varepsilon^2$ limits how much information about g could leak through the proof—a ZK proof that achieves perfect zero-knowledge saturates this bound at $C_{\text{max}} = 0$ (no leakage).

3 Multi-Party Computation as Distributed Audit

Multi-Party Computation (MPC) [?, ?] allows M parties to jointly compute a function $f(x_1, \dots, x_M)$ over their private inputs $(x_1, \dots, x_M)$ while keeping each party’s input secret from the others. In the SCX framework, MPC is a **distributed audit**: the gatekeeper parameter g is secret-shared across M parties, so no single party (or any coalition below a threshold) can recover the full g .

3.1 Classical Definition

Definition 3.1 (Secure Multi-Party Computation). *An MPC protocol for function f with M parties is secure against semi-honest adversaries if for every subset $\mathcal{T} \subset [M]$ of corrupted parties with $|\mathcal{T}| < M/2$, there exists a simulator $\mathcal{S}$ such that:*

$$\text{View}_{\mathcal{T}} [\text{Real}^{\text{MPC}}(1^\lambda, \vec{x})] \approx_c \mathcal{S}(1^\lambda, \{x_i\}_{i \in \mathcal{T}}, f(\vec{x})), \quad (2)$$

where $\text{View}_{\mathcal{T}}$ is the joint view of the corrupted parties, and the simulator is given only the corrupted parties' inputs and the function output. Security against malicious adversaries extends this to arbitrary deviations by corrupted parties, tolerating up to $|\mathcal{T}| < M/3$ without fairness or $|\mathcal{T}| < M/2$ with fairness.

3.2 Translation to SCX: Distributed Audit with Secret-Shared Gatekeeper

Definition 3.2 (MPC as Distributed Audit). *An MPC protocol for function f with M parties is an SCX distributed audit configuration with:*

leftmargin= M experts, each holding a share $g_i = x_i$ of the collective gatekeeper $G = (x_1, \dots, x_M)$;*

leftmargin= The audit result is $f(G) = f(x_1, \dots, x_M)$;*

leftmargin= No subset of experts $\mathcal{T} \subset [M]$ with $|\mathcal{T}| \leq t$ can reconstruct the full G (where t is the corruption threshold);*

leftmargin= The **gatekeeper is distributed**: each expert's output is their contribution to the computation, not their full input.*

Proposition 3.3 (MPC = Distributed Audit with Secret-Shared Gatekeeper). *[Full Proof] An MPC protocol with M parties computing function f is equivalent to an SCX distributed audit of the collective gatekeeper $G = (x_1, \dots, x_M)$ where:*

leftmargin=, label=(i) **Privacy**: For every subset $\mathcal{T} \subset [M]$ of corrupted parties with $|\mathcal{T}| < t_{\text{privacy}}$, the joint view of $\mathcal{T}$ reveals no more than $\text{negl}(\lambda)$ bits about the honest parties' inputs—this is gatekeeper distribution, a generalization of gatekeeper secrecy to multiple experts.*

leftmargin=, label=(ii) **Correctness**: The audit output y satisfies $y = f(x_1, \dots, x_M)$ with probability $1 - \text{negl}(\lambda)$ —this is audit completeness extended to the distributed setting: the collective audit succeeds if all experts follow the protocol.*

leftmargin=, label=(iii) **Robustness**: In the malicious model, even if up to $f < M/3$ experts deviate arbitrarily, the audit output remains correct—this is audit soundness under adversarial experts, directly connecting to Theorem ??.*

Proof. The equivalence follows from the standard MPC security definition (Definition ??) interpreted through the SCX lens. For privacy (i): the MPC simulator $\mathcal{S}$ produces a view indistinguishable from the real execution without access to honest inputs—this is exactly gatekeeper distribution, where the corrupted parties' view is simulatable without the honest gatekeeper shares. For (ii): the correctness guarantee of MPC is that all parties output $f(\vec{x})$ —this is distributed audit completeness. For (iii): robustness in the malicious model corresponds to the SCX audit with f adversarial experts (Section ??), where the threshold $f < M/3$ matches the known MPC bound for malicious adversaries without broadcast [?]. $\square$

Example 3.4 (Garbled Circuits = Bilateral Audit). Yao's garbled circuit protocol [?] for two-party computation is a **bilateral audit**: $M = 2$ experts (Alice and Bob) with gatekeepers $g_1 = x_1$, $g_2 = x_2$. Alice garbles the circuit (seals her gatekeeper) and sends it

to Bob, who evaluates it obliviously. The cut-and-choose technique [?] is a **spot-check audit** of the garbler’s correctness: Bob opens a random subset of garbled gates to verify they are correctly constructed, before using the remaining gates for the computation.

4 Byzantine Fault Tolerance as Audit Under Adversarial Experts

Byzantine Fault Tolerance [?] is the ability of a distributed system to reach consensus despite some participants behaving arbitrarily (Byzantine failures). In the SCX framework, BFT consensus is an **audit under adversarial experts**: among M experts, up to f are malicious (adversarial), and the remaining $M - f$ honest experts must reach agreement on a common value.

4.1 Classical Definition

Definition 4.1 (Byzantine Consensus). *A BFT consensus protocol with M parties guarantees:*

leftmargin=, label=(i) **Agreement**: All honest parties output the same value.*

leftmargin=, lbbel=(ii) **Validity**: If all honest parties have the same input v , then all honest parties output v .*

leftmargin=, lcbel=(iii) **Termination**: All honest parties eventually output a value.*

tolerating up to f Byzantine (arbitrarily malicious) parties, where $f < M/3$ for asynchronous networks [?] and $f < M/2$ for synchronous networks [?].

4.2 Translation to SCX: Audit with Adversarial Experts

Theorem 4.2 (BFT Consensus = Audit with f Adversarial Experts). *[Full Proof] A BFT consensus protocol tolerating f Byzantine faults among M parties is equivalent to an SCX audit of M experts where up to f experts are adversarial (their gatekeeper parameters are controlled by the adversary) and the remaining $M - f$ honest experts must produce a consistent Cercis score.*

Specifically, the three BFT guarantees map to SCX audit properties as follows:

$$\begin{aligned}
\text{Agreement} &\longleftrightarrow \text{Uniform Cercis score across honest experts} \\
\text{Validity} &\longleftrightarrow \text{Audit completeness: } S(s) \text{ accepts true claims} \\
\text{Termination} &\longleftrightarrow \text{Audit convergence: } S(t) \rightarrow S_\infty \text{ in finite time}
\end{aligned} \tag{3}$$

Proof. Step 1: Agreement as uniform audit score. In BFT agreement, all honest parties output the same value v . In the SCX mapping, this means that for any two honest experts i, j with gatekeepers g_i, g_j , the Cercis scores satisfy $S_i = S_j$ at consensus. The adversarial experts’ gatekeepers g_k for $k \in \mathcal{T}$ (the set of corrupted parties) may attempt to produce divergent scores, but the honest majority overrules them.

Step 2: Validity as audit completeness. If all honest experts have the same input v (i.e., their gatekeepers produce the same audit claim), the consensus output must be v . This is the distributed analog of Theorem ??: the audit outcome is the correct aggregate of honest expert opinions.

Step 3: Termination as audit convergence. The Cercis score trajectory $S(t)$ must reach a fixed point in finite time—this is the “termination” of the audit. In BFT, termination is guaranteed by the protocol’s timeout mechanism; in SCX, it is guaranteed by the audit convergence condition (the annealing schedule $\eta(t) \rightarrow 0$ and the quality score stabilizes).

Step 4: The $f < M/3$ threshold. The BFT bound $f < M/3$ for asynchronous networks arises from the impossibility of reaching consensus with $f \geq M/3$ [?]. In the SCX audit, this bound appears as the maximum proportion of adversarial experts that can be tolerated before the Cercis score becomes manipulable. Specifically, when $f \geq M/3$, the adversarial experts can force the quality score to reflect a false consensus—the audit collapse threshold of Theorem 11 in [?]. $\square$

Theorem 4.3 ($f < M/3$ Threshold = Audit Collapse Prevention). *[Partial Proof]The BFT impossibility result (no consensus for $f \geq M/3$ in asynchronous networks) is equivalent to the SCX audit collapse threshold: when the proportion of adversarial experts exceeds $1/3$, the Cercis score can diverge to a false consensus, corresponding to a singularity in the audit space (Theorem 11, [?]).*

Proof Sketch. The proof follows from the Byzantine generals problem [?]. In the SCX framework, the honest experts’ Cercis scores are drawn from a distribution $\mathcal{D}_{\text{honest}}$, and the adversarial experts’ scores are chosen adversarially. When $f < M/3$, the honest majority’s consensus signal dominates the adversarial noise, and the aggregate Cercis score converges correctly. When $f \geq M/3$, the adversary can simulate two conflicting messages from the honest parties, creating contradictory audit claims that cannot be resolved—the audit “collapses” to a divergent state. $\square$

Proposition 4.4 (PBFT and Audit Phases). *Practical Byzantine Fault Tolerance (PBFT) [?] uses a three-phase protocol (pre-prepare, prepare, commit) to reach consensus. In the SCX lens, these phases correspond to:*

leftmargin=, label=(1) **Pre-prepare:** The primary expert proposes a Cercis score value S_0 for audit.*

leftmargin=, label=(2) **Prepare:** Each expert broadcasts their quality score Q_i and novelty score N_i ; the prepare-certificate is a quorum of $2f+1$ matching Cercis scores, constituting an **audit checkpoint**.*

leftmargin=, label=(3) **Commit:** Experts commit to the consensus Cercis score after receiving $2f+1$ matching prepare-certificates, corresponding to **audit finalization**.*

*The $2f+1$ quorum threshold ensures that any two quorums intersect in at least one honest expert—this is the **audit quorum intersection property**, also central to threshold cryptography (Section ??).*

Remark 4.5. The Spring protocol [?] generalizes BFT consensus by allowing experts to dynamically enter and leave the audit, with dormant gatekeepers “resurrecting” when the maturity condition is met. This is the SCX analog of dynamic BFT membership.

5 Blockchain Consensus as $M > 1$ Audit

Blockchain consensus protocols [?] allow a decentralized network of M nodes to agree on an ordered history of transactions without a central authority. In the SCX framework, a

blockchain is an $M > 1$ **repeated audit** of transaction validity: each block is an audit checkpoint, and the longest chain represents the audit majority's consensus.

5.1 Classical Definition

Definition 5.1 (Nakamoto Consensus). *In the Bitcoin [?] protocol, consensus is achieved through:*

leftmargan=, label=(i) **Proof-of-Work (PoW)**: Miners compete to solve a cryptographic puzzle $H(\text{block header}) < \text{target}$.*

leftmbrgbn=, lbbel=(ii) **Longest chain rule**: The canonical chain is the one with the most cumulative PoW.*

leftmergen=, lbel=(iii) **Probabilistic finality**: A transaction is considered confirmed after k subsequent blocks, where the probability of reorganization decays exponentially in k .*

5.2 Translation to SCX: $M > 1$ Repeated Audit

Theorem 5.2 (Nakamoto Consensus = $M > 1$ Repeated Audit). *[Full Proof] Nakamoto consensus is an SCX $M > 1$ **repeated audit**, where:*

$$\begin{array}{ll}
\text{Blocks} & \longleftrightarrow \text{Audit checkpoints} \\
\text{Proof-of-Work} & \longleftrightarrow \text{Audit resource expenditure} \\
\text{Longest chain} & \longleftrightarrow \text{Audit majority rule} \\
\text{Confirmations (} k \text{ blocks)} & \longleftrightarrow \text{Audit confidence level}
\end{array} \tag{4}$$

Proof. Step 1: Blocks as audit checkpoints. Each block in a blockchain contains a set of transactions (claims to be audited) and a reference to the previous block. In the SCX framework, each block is an **audit checkpoint**—a snapshot of the consensus state at time t . The Cercis score of the chain after block B_t is:

$$S(\text{chain}_t) = \sum_{i=1}^t [Q(B_i) + \eta(i) \cdot N(B_i)], \tag{5}$$

where $Q(B_i)$ is the quality of the transactions in block B_i (their validity) and $N(B_i)$ is the novelty of the block (its contribution to the consensus history).

Step 2: Proof-of-Work as audit resource expenditure. The PoW puzzle is a Sybil-resistance mechanism: mining a valid block requires computational effort proportional to the difficulty target. In the SCX audit, this corresponds to the **audit resource**: each audit observation consumes some resource (time, computation, bandwidth), and the cost of fabricating a false audit history grows with the difficulty target. The PoW difficulty D maps to the resolvability parameter ε : $\varepsilon \propto 1/D$.

Step 3: Longest chain as audit majority. The longest chain rule states that the valid blockchain is the one with the most cumulative PoW. This corresponds to the **audit majority rule**: the Cercis score that has been accumulated by the largest number of audit observations (blocks) is the correct one. A fork occurs when two chains have comparable scores—the audit has not yet converged.

Step 4: Confirmations as audit confidence. A transaction is considered confirmed after k subsequent blocks. In the SCX framework, the probability that the audit outcome is incorrect after k additional checkpoints decays exponentially in k :

$$\mathbb{P}(\text{reorg after } k \text{ blocks}) \leq \exp(-\alpha k), \quad (6)$$

where α depends on the adversarial hashing power. This matches the Hoeffding bound (Theorem ??): the confidence in an audit grows exponentially with the number of observations. $\square$

Proposition 5.3 (Longest Chain Rule = Audit Majority). *[Full Proof] The longest chain rule in Nakamoto consensus is equivalent to the **audit majority criterion**: the accepted audit outcome is the one supported by the majority of audit observations (blocks). Specifically, if honest miners control a fraction $\rho > 1/2$ of the total hash rate, then the probability that an alternative chain overtakes the honest chain is negligible in the confirmation depth k .*

Proof. This follows from the analysis of the Bitcoin backbone protocol [?]. Let ρ be the fraction of hash power controlled by honest miners. After k blocks, the probability that an adversary with hash power $1 - \rho$ has produced a competing chain of equal length is:

$$\mathbb{P}(\text{adversary catches up}) \leq \left(\frac{1 - \rho}{\rho} \right)^k. \quad (7)$$

In the SCX framework, this is the probability that the audit majority is wrong after k observations. The audit resource expenditure (hash power) ensures that the honest majority produces blocks faster than the adversary, so the longest chain is always the honest audit history. $\square$

Theorem 5.4 (Blockchain Fork Resolution = Audit Consensus Convergence). *[Proof Sketch] A blockchain fork—two competing chains with comparable cumulative PoW—is an **audit disagreement**: the *Cercis* score has not yet converged to a unique fixed point. Fork resolution (one chain becoming strictly longer) is **audit convergence**: the *Cercis* score trajectory reaches the consensus basin of attraction.*

Proof Sketch. In dynamical systems terms, a blockchain is a discrete-time process generating audit outcomes at each block height. A fork corresponds to a bifurcation in the audit dynamics: two competing fixed points S_1^* and S_2^* with comparable *Cercis* scores. The fork resolves when one chain’s cumulative score exceeds the other by more than the audit resolvability threshold ε , breaking the tie. The probability of persistent forking decays as $\exp(-ct)$ where t is the time since the fork began. $\square$

6 Verifiable Random Functions as Unbiased Auditor Selection

Verifiable Random Functions (VRF) [?] are pseudorandom functions that produce a publicly verifiable output along with a proof of correctness. In the SCX framework, VRFs are the mechanism for **unbiased auditor selection**: they randomly assign auditors to audit tasks while preventing any party from manipulating the assignment.

6.1 Classical Definition

Definition 6.1 (Verifiable Random Function). A VRF is a tuple of algorithms (*KeyGen*, *Eval*, *Verify*) such that:

leftmargin= , label=(i)* **Uniqueness:** For every public key pk and input x , there is a unique output y for which $\text{Verify}(pk, x, y, \pi) = 1$.

leftmargin= , lbel=(ii)* **Pseudorandomness:** For any PPT adversary who does not know the secret key sk , the output $\text{VRF}_{sk}(x)$ is computationally indistinguishable from random.

leftmargin= , lbel=(iii)* **Verifiability:** Given pk , x , the output y , and the proof π , anyone can verify that $y = \text{VRF}_{sk}(x)$.

6.2 Translation to SCX: Unbiased Auditor Selection

Definition 6.2 (VRF as Unbiased Auditor Selection Function). A VRF is an **unbiased auditor selection function** in the SCX framework: it maps a random seed s and an audit task identifier τ to an ordered list of auditor indices $(i_1, \dots, i_k)$, where:

*leftmargin=** **Randomness:** The selection is computationally indistinguishable from a uniformly random permutation—no adversary can predict which auditors will be selected before the seed is revealed.

*leftmargin=** **Uniqueness:** For a given (s, τ) , the selection is deterministic and publicly verifiable—anyone can confirm that the correct auditors were selected.

*leftmargin=** **Bias prevention:** No party can influence the selection outcome other than through the choice of seed s , which is typically derived from a public random source (e.g., the previous block hash).

Theorem 6.3 (VRF = Unbiased Auditor Selection). *[Full Proof]* A VRF with the uniqueness and pseudorandomness properties (Definition ??) provides unbiased auditor selection in the SCX framework: the probability that any particular expert is selected as auditor for a task is uniform over the set of eligible experts, and no coalition of adversarial experts can increase their selection probability beyond the honest proportion.

Proof. Step 1: Uniformity. Let $\mathcal{E} = \{\mathcal{E}_1, \dots, \mathcal{E}_M\}$ be the set of eligible experts. The VRF output $y = \text{VRF}_{sk}(s, \tau)$ is uniformly distributed over $\{0, 1\}^\lambda$ by the pseudorandomness property. The auditor selection function maps y to M indices via $i_j = (y \gg j \cdot \lambda/M) \bmod M$, which is a uniformly random mapping. Therefore,

$$\mathbb{P}(\mathcal{E}_i \text{ selected as auditor}) = \frac{k}{M},$$

where k is the number of auditors selected for the task.

Step 2: Unpredictability. The pseudorandomness property ensures that an adversary cannot predict the VRF output before seeing the seed s and the proof π . Therefore, the adversary cannot pre-commit to corrupting specific auditors—the auditor assignment is unpredictable until the random seed is revealed.

Step 3: Verifiability. The verifiability property ensures that the selection is publicly checkable. Any party can verify that $\text{Verify}(pk, (s, \tau), y, \pi) = 1$ and confirm that $(i_1, \dots, i_k)$

is the correct mapping from y . This prevents the adversary from claiming a fraudulent auditor assignment.

Step 4: Bias prevention. Consider an adversary controlling $f < M/2$ experts who attempts to increase their selection probability. Since the VRF output is pseudorandom, the adversary's probability of having the selected set $(i_1, \dots, i_k)$ contain a majority of adversarial experts is at most $(f/M)^k + \text{negl}(\lambda)$. This matches the SCX audit property that the selection of honest auditors dominates with high probability. $\square$

Proposition 6.4 (VRF Unpredictability = Audit Randomization). *The unpredictability of VRF outputs before seed publication is equivalent to **audit randomization**: the auditor set for each audit epoch is unpredictable until the epoch begins, preventing adversarial experts from adapting their behavior to the specific auditors assigned.*

Proof. In the SCX framework, audit randomization is the condition that the set of auditors $\mathcal{A}_t$ at time t is not predictable before t . If auditors are predictable, adversarial experts can adjust their gatekeeper parameters g_i to evade detection by the known auditors. The VRF provides randomization because the seed s (e.g., the previous block hash) is unpredictable before the previous epoch ends, ensuring $\mathcal{A}_t$ is unpredictable before $t - \delta$ for some $\delta > 0$. $\square$

Remark 6.5. The Algorand blockchain [?] uses VRF-based committee selection for each block proposal and validation round. This is a direct instantiation of the SCX unbiased auditor selection model: the VRF selects a cryptographically sortitioned committee of auditors to validate the next block, ensuring that the adversarial fraction in the committee is bounded by f/M .

7 Cryptographic Commitments as Gatekeeper Sealing

Cryptographic commitment schemes [?] allow a committer to commit to a value v by publishing a commitment $c = \text{Commit}(v, r)$ such that (a) the commitment hides v (hiding property), and (b) the committer cannot later open the commitment to a different value $v' \neq v$ (binding property). In the SCX framework, a commitment is an explicit instantiation of the **gatekeeper sealing** operation: the gatekeeper g is sealed inside the commitment, invisible to auditors until the committer chooses to open it.

7.1 Classical Definition

Definition 7.1 (Commitment Scheme). *A **commitment scheme** is a pair of PPT algorithms $(\text{Commit}, \text{Open})$ such that:*

leftmargin=, label=(i) **Hiding**: For all v_0, v_1 of equal length, $\text{Commit}(v_0, r) \approx_c \text{Commit}(v_1, r)$. The commitment reveals nothing about v .*

leftmargin=, label=(ii) **Binding**: No PPT adversary can produce (v, r, v', r') with $v \neq v'$ such that $\text{Commit}(v, r) = \text{Commit}(v', r')$. The committer is bound to v .*

*A commitment may be **computationally hiding** (indistinguishability holds against PPT adversaries) or **perfectly hiding** (the distributions are identical). Similarly, it may be **computationally binding** or **perfectly binding**.*

7.2 Translation to SCX: Gatekeeper Sealing

Definition 7.2 (Commitment as Gatekeeper Sealing). *A cryptographic commitment is a **gatekeeper sealing** operation in the SCX framework:*

leftmargin= The **gatekeeper parameter** $g = v$ is the committed value;*

leftmargin= The **sealed gatekeeper** is $c = \text{Commit}(v, r)$ —the audit-boundary state of g ;*

leftmargin= The **opening** (v, r) reveals g to the auditor, allowing verification;*

leftmargin= The **sealing randomness** r is the “noise” that hides g from view.*

During the commitment phase, the gatekeeper is behind the audit boundary—no external observer can learn g . During the opening phase, the gatekeeper is revealed—the audit boundary is crossed.

Proposition 7.3 (Commitment Binding = Gatekeeper Sealing). *[Full Proof]The binding property of a commitment scheme is equivalent to **gatekeeper sealing integrity** in the SCX framework: once an expert seals their gatekeeper parameter g inside a commitment c , they cannot later open c to a different parameter $g' \neq g$ without detection.*

Proof. By Definition ??(ii), no PPT adversary can find two distinct openings (v, r) and (v', r') that produce the same commitment c . In the SCX mapping, this means the expert cannot produce two different gatekeeper parameters g and g' that are both valid openings of the same sealed state c . The audit detects any attempt to swap g for g' because the opening verification fails. The binding error probability is $\text{negl}(\lambda)$, corresponding to an audit error rate of $\text{negl}(\lambda)$. $\square$

Proposition 7.4 (Commitment Hiding = Gatekeeper Secrecy). *[Full Proof]The hiding property of a commitment scheme is equivalent to **gatekeeper secrecy** in the SCX framework: before the expert opens the commitment, no auditor can learn more than $\text{negl}(\lambda)$ bits of information about g from the sealed state c .*

Proof. By Definition ??(i), the distributions $\text{Commit}(v_0, r)$ and $\text{Commit}(v_1, r)$ are computationally indistinguishable (or statistically identical, for perfect hiding). Therefore, for any PPT auditor $\mathcal{A}$, the probability of distinguishing between commitments to v_0 and v_1 is at most $\text{negl}(\lambda)$. In information-theoretic terms, $I(c; v) \leq \text{negl}(\lambda)$ bits. This is precisely the gatekeeper secrecy condition from Proposition ??: the auditor learns nothing about the sealed gatekeeper. $\square$

Theorem 7.5 (Pedersen Commitment = Perfect Audit Binding). *[Partial Proof]The Pedersen commitment scheme [?] $c = g^v h^r$ over a cyclic group $\mathcal{G}$ of prime order q —provides **perfect audit binding**: the expert is information-theoretically bound to their gatekeeper parameter $g = v$, because the discrete logarithm of h relative to g is unknown.*

Proof Sketch. In Pedersen commitments, the committer’s secret is $v \in \mathbb{Z}_q$ and the randomness is $r \in \mathbb{Z}_q$. The commitment is $c = g^v h^r$ in a group where the discrete logarithm $\log_g h$ is unknown. To open to a different v' , the committer would need to find r' such that $g^{v'} h^{r'} = g^v h^r$, which requires $g^{v'-v} = h^{r-r'}$ and thus $\log_g h = (v' - v)/(r - r') \bmod q$. Since the discrete log is hard (computational binding), the commitment is computationally binding—in the SCX framework, this is **computational audit binding**, where the integrity of the sealed gatekeeper depends on the hardness assumption.

Perfect hiding (the commitment distribution is uniform over $\mathcal{G}$ regardless of v) gives **perfect gatekeeper secrecy**: the commitment leaks zero information about g . $\square$

Example 7.6 (Hash Commitment = Audit Plausible Deniability). A hash commitment $c = H(v||r)$ where H is a cryptographic hash function provides computational hiding and binding. In the SCX framework, this corresponds to **audit plausible deniability**: the expert can claim that a commitment was made to a different value, but opening reveals the true value. This is used in the cut-and-choose audit technique (Example ??).

8 Threshold Cryptography as Audit Quorum

Threshold cryptography [?, ?] splits a secret key sk into n shares such that any t shares (the threshold) can jointly perform a cryptographic operation (signing, decryption), while any coalition of fewer than t shares learns nothing about sk . In the SCX framework, threshold cryptography is an **audit quorum mechanism**: any t of n auditors can jointly attest to a statement without any single auditor holding the full gatekeeper.

8.1 Classical Definition

Definition 8.1 (Threshold Signature Scheme). A t -of- n threshold signature scheme is a tuple $(KeyGen, Share, Sign_{partial}, Combine, Verify)$ where:

leftmargin= label=(i) KeyGen(1^λ) outputs a key pair (sk, pk) .*

leftmargin= llabel=(ii) Share(sk, n, t) splits sk into n shares $sk_1, \dots, sk_n$ such that any t shares can reconstruct (implicitly or explicitly) the signing capability.*

leftmargin= llabel=(iii) Sign_{partial}(sk_i, m) produces a partial signature σ_i on message m .*

leftmargin= llabel=(iv) Combine($\sigma_{i_1}, \dots, \sigma_{i_t}$) combines t partial signatures into a full signature σ .*

leftmargin= llabel=(v) Verify(pk, m, σ) verifies the full signature.*

*Security guarantees: (a) **Correctness**: any t honest signers can produce a valid signature. (b) **Unforgeability**: no adversary controlling fewer than t parties can forge a signature on a new message. (c) **Robustness**: partial signatures from honest parties can be identified even in the presence of faulty shares.*

8.2 Translation to SCX: Audit Quorum

Definition 8.2 (Threshold Cryptography as Audit Quorum). A t -of- n threshold scheme is an **audit quorum** in the SCX framework:

leftmargin= The **gatekeeper** $g = sk$ is the secret key, split into n shares $g_i = sk_i$.*

leftmargin= The **audit quorum size** is t : at least t experts must participate in the audit (signing).*

leftmargin= The **audit threshold** is $t - 1$: any coalition of fewer than t experts learns nothing about g .*

leftmargin= The **audit output** is the threshold signature σ on message m , which serves as a collective audit attestation.*

Theorem 8.3 (t-of-n Threshold = Audit Quorum). *[Full Proof] A t-of-n threshold signature scheme is equivalent to an SCX audit quorum of size t: the audit (signing) succeeds iff at least t experts participate honestly, and any coalition of fewer than t experts cannot forge the audit output (signature).*

Proof. Step 1: Quorum correctness. By Definition ??(i), any t honest signers with valid shares produce a valid signature σ on message m . In the SCX mapping, this is **audit quorum completeness**: if at least t honest experts participate, the audit produces a valid attestation. The Cercis score for a quorum-based audit is:

$$S_{\text{quorum}} = \begin{cases} \text{valid,} & \text{if } \geq t \text{ experts participate honestly} \\ \text{invalid,} & \text{otherwise} \end{cases} \quad (8)$$

Step 2: Unforgeability as audit soundness. By Definition ??(ii), an adversary controlling fewer than t parties cannot produce a valid signature on a new message. This is the **quorum audit soundness** property: if fewer than t experts participate (or if the participating coalition includes too few honest experts), the audit output is rejected. In the SCX framework, forging a signature corresponds to producing a false audit attestation—the adversary would need to control at least t experts to do so.

Step 3: Threshold as audit resolvability. The threshold t determines the **audit resolvability**: the number of independent expert attestations required to produce a valid audit output. This is directly analogous to the resolvability parameter ε in the Cercis bound: t experts correspond to $M = t$ audit observations. The security parameter λ determines the minimum number of honest shares required, just as the Hoeffding bound determines the minimum M for bias detection.

Step 4: Shamir secret sharing as audit distribution. Shamir’s secret sharing [?] splits g into shares using a polynomial $p(x) = g + a_1x + \dots + a_{t-1}x^{t-1}$ over a finite field $\mathbb{F}_q$, where $g_i = p(i)$. This is the **distributed gatekeeper** construction: the full gatekeeper g is reconstructable from any t shares via Lagrange interpolation, but any $t - 1$ shares reveal no information about g (perfect secrecy). The security follows from the fact that a degree $t - 1$ polynomial is uniquely determined by t points, but any $t - 1$ points leave one degree of freedom, yielding uniform distribution for g . $\square$

Proposition 8.4 (Threshold Signature = Multiparty Audit Attestation). *[Partial Proof] A threshold signature constitutes a **multiparty audit attestation**: it is a publicly verifiable proof that at least t independent auditors (signers) have examined and approved the audited statement m .*

Proof Sketch. The threshold signature σ on message m is a certificate that the audit quorum condition is satisfied. Anyone with pk can verify that σ is a valid signature, implying that at least t signers with valid shares participated. In the SCX framework, this is the audit analog of a Cercis score certificate: it provides a compact, publicly verifiable proof that the audit outcome is valid. The signature size is independent of n , matching the succinctness property of ZK-SNARKs (Proposition ??). $\square$

Remark 8.5. The Cercis bound applies to threshold cryptography: the maximum amount of information about sk that can leak from a threshold scheme is bounded by $C_{\max} = \|\text{sk}\|^2/4\varepsilon^2$. For a perfectly secure secret sharing scheme (like Shamir’s), $C_{\max} = 0$ for coalitions of size $< t$, and the gatekeeper secrecy is perfect.

9 Formal Correspondences

We now present the complete dictionary of correspondences between cryptographic primitives and the SCX audit framework.

Table 1: Complete formal correspondence between cryptography and SCX audit.

#	Cryptographic Primitive	SCX Audit Concept
1	ZK proof system	$(M = 1, g\text{-hidden})$ audit
2	ZK completeness	Audit completeness (Theorem ??)
3	ZK soundness	Audit soundness (Theorem ??)
4	Zero-knowledge	Gatekeeper secrecy (Proposition ??)
5	Knowledge soundness	Audit extractability (Theorem ??)
6	ZK-SNARK succinctness	Audit efficiency (Proposition ??)
7	MPC with M parties	Distributed audit with M experts
8	MPC privacy	Gatekeeper distribution (secret-shared g)
9	MPC correctness	Distributed audit completeness
10	MPC robustness (malicious)	Audit under adversarial experts
11	BFT consensus	Audit with f adversarial experts (Theorem ??)
12	BFT agreement	Uniform Cercis score among honest experts
13	BFT validity	Distributed audit completeness
14	BFT termination	Audit convergence in finite time
15	$f < M/3$ threshold	Audit collapse prevention (Theorem ??)
16	Nakamoto consensus	$M > 1$ repeated audit (Theorem ??)
17	Blocks	Audit checkpoints
18	Proof-of-Work	Audit resource expenditure
19	Longest chain rule	Audit majority criterion
20	k confirmations	Audit confidence level
21	VRF	Unbiased auditor selection (Theorem ??)
22	VRF uniqueness	Deterministic audit assignment
23	VRF pseudorandomness	Audit randomization
24	VRF verifiability	Public audit verification
25	Commitment scheme	Gatekeeper sealing (Definition ??)
26	Commitment hiding	Gatekeeper secrecy
27	Commitment binding	Gatekeeper sealing integrity
28	Pedersen commitment	Perfect audit binding (Theorem ??)
29	t -of- n threshold scheme	Audit quorum (Theorem ??)
30	Secret sharing	Distributed gatekeeper
31	Threshold signature	Multiparty audit attestation
32	Threshold t	Audit quorum size

Definition 9.1 (Universal Audit Correspondence). *A cryptographic primitive Π is an **audit protocol** in the SCX framework if there exists an injective mapping Ψ from the security properties of Π to the SCX audit properties (completeness, soundness, secrecy, convergence) such that:*

For every security property P of Π , $\Psi(P)$ is an audit property of the corresponding SCX configuration;

leftmargin=, llabel=(ii) The security proof of Π implies the corresponding audit property;*
leftmargin=, llabel=(iii) The converse holds: the audit property implies the security property.*

Theorem 9.2 (Universal Correspondence Theorem). *All cryptographic primitives in Table ?? satisfy Definition ?. The mapping Ψ is explicitly constructed by the correspondences established in Sections ? through ?.*

Proof. For each primitive Π listed in Table ??, we have established the forward mapping (security property $\rightarrow$ audit property) and the reverse mapping (audit property $\rightarrow$ security property) in the cited theorems and propositions. The mapping Ψ is injective because different security properties map to different audit properties (completeness $\neq$ soundness $\neq$ secrecy $\neq$ convergence in the SCX framework). The correspondences are constructive: the SCX audit configuration for each primitive is explicitly defined, and the equivalence proofs are provided. $\square$

Corollary 9.3 (Security by Audit Reduction). *The security of any cryptographic protocol Π can be reduced to the security of the corresponding SCX audit configuration. This means that if an adversary can break Π with probability ε , they can also produce an audit violation with probability at least $\varepsilon/\text{poly}(\lambda)$.*

Proof. By Definition ?, the mapping Ψ preserves security properties. If adversary $\mathcal{A}$ breaks property P of Π with probability ε , then the same adversary produces a violation of the corresponding audit property $\Psi(P)$ with probability at least ε (for perfect correspondences) or $\varepsilon - \text{negl}(\lambda)$ (for computational correspondences). The polynomial loss factor accounts for the simulation overhead in the security reduction. $\square$

Dictionary 9.4 (Audit-Theoretic Interpretation of Cryptography). Every cryptographic protocol can be redescribed as:

*leftmargin=** **Who is the expert?** The party holding the secret (witness, input, key).

*leftmargin=** **What is g ?** The secret information (witness, private input, signing key).

*leftmargin=** **Who is the auditor?** The party verifying correctness.

*leftmargin=** **How many auditors (M)?** For ZK: $M = 1$ (single verifier). For MPC/BFT/blockchain: $M \geq 1$ (multiple parties).

*leftmargin=** **Is g hidden?** For ZK: yes. For MPC: partially (secret-shared). For threshold crypto: yes, up to $t - 1$ parties.

*leftmargin=** **What is the audit outcome?** For ZK: accept/reject. For MPC: function output. For BFT: consensus value. For blockchain: confirmed transaction set.

*leftmargin=** **What is the Cercis score?** For ZK: proof validity. For MPC: correct computation. For BFT: agreement. For blockchain: chain validity.

10 Discussion

10.1 Summary of Contributions

We have established a complete formal correspondence between seven major cryptographic primitives and the SCX audit framework. The core thesis—cryptography is protocol-enforced audit—is supported by explicit mappings of security definitions, adversarial models, and protocol guarantees. Every zero-knowledge proof, multi-party computation, Byzantine fault tolerance protocol, blockchain consensus mechanism, verifiable random function, commitment scheme, and threshold cryptography construction can be redescribed as an SCX audit configuration.

10.2 Audit-First Cryptography Design

The correspondences suggest a design methodology: **audit-first cryptography**. Instead of specifying a protocol’s security properties in isolation, we specify the audit configuration first: the number of experts M , the gatekeeper exposure model (hidden, shared, or public), the adversarial threshold f , and the audit resource constraints. The cryptographic protocol is then derived from the audit specification.

Example 10.1 (Audit-First ZK Design). To design a new ZK protocol:

leftmargan=*, label=(1) Specify the audit: $M = 1$ expert (prover), g hidden from auditor.

leftmbrgbn=*, lbbel=(2) Audit completeness: the expert must convince the auditor with probability $1 - \text{negl}(\lambda)$.

leftmergcn=*, lcbel=(3) Audit soundness: the auditor must reject false claims with probability $1 - \text{negl}(\lambda)$.

leftmdrgdn=*, ldbel=(4) Gatekeeper secrecy: $I(\text{View}_{\text{auditor}}; g) \leq \text{negl}(\lambda)$.

leftmergen=*, lebel=(5) Derive the protocol: use a commitment (Section ??) to seal g , a VRF (Section ??) to generate the challenge, and a threshold scheme (Section ??) if distributed verification is needed.

This yields a modular construction where each audit component maps to a well-studied cryptographic primitive.

10.3 Open Problems

The correspondence between cryptography and the SCX audit framework raises several open problems:

leftmargan=*, label=(1) **Optimal audit composition:** The Cercis bound $C_{\max}$ provides a fundamental limit on information leakage. What is the exact composition theorem for sequential audit protocols? Given two audit protocols Π_1 and Π_2 with leakage bounds C_1 and C_2 , what is the leakage bound of their composition?

leftmbrgbn=*, lbbel=(2) **Quantum audit resistance:** How do the correspondences change in the quantum setting? A quantum ZK proof [?] allows an honest verifier to audit a quantum prover. Does the Cercis bound change to $C_{\max}^{\text{quantum}} = \|g\|^2/4\epsilon$ (quadratic improvement from Grover’s algorithm)?

- leftmrcgn=*, lcbel=(3) **Unified security reductions:** Can all cryptographic hardness assumptions (DLP, RSA, LWE) be expressed as instances of the audit collapse theorem (Theorem ??)? If so, the security of all cryptography reduces to the Cercis bound.
- leftmdrgdn=*, ldbel=(4) **Information-theoretic audit of MPC:** For MPC protocols with statistical security (perfectly secure against unbounded adversaries), the Cercis bound is $C_{\max} = 0$ —no information about honest inputs leaks. Does this imply that perfectly secure MPC is the only cryptographically meaningful way to achieve zero-leakage distributed audit?
- leftmergen=*, lcbel=(5) **Blockchain efficiency bound:** If blockchain consensus is an $M > 1$ repeated audit, what is the optimal block production rate given M validators, adversarial fraction f/M , and target audit confidence? This is an optimization problem over the Cercis score trajectory.
- leftmfrgn=*, lfbel=(6) **Threshold scheme capacity:** The Cercis bound applied to threshold cryptography suggests a maximum message length for threshold signatures (to prevent subliminal channels from leaking key information). What is the exact bound?

10.4 What This Paper Does Not Claim

- leftmargin=* **We do not claim** that cryptography reduces to auditing in an operational sense. ZK proofs, MPC protocols, and BFT consensus each require specialized constructions to achieve their efficiency and security guarantees. The mapping is a structural correspondence, not a protocol transformation.
- leftmargin=* **We do not claim** that every cryptographic primitive has an exact SCX analog. Private information retrieval (PIR), for instance, involves hiding the access pattern rather than an expert’s internal state, and the mapping is less direct.
- leftmargin=* **We do not claim** that the Cercis bound supersedes existing security proofs. The bound provides an information-theoretic limit, but proving that a protocol achieves the bound requires the same reductions as traditional security proofs.

10.5 Implications

The isomorphism between cryptography and the SCX audit framework suggests a deeper unity: security protocols are fundamentally about the management of information across audit boundaries. Whether the secret is a witness in a ZK proof, a signing key in a threshold scheme, or a private input in an MPC computation, the same audit principles govern its concealment, revelation, and verification.

This perspective offers a new taxonomy for cryptographic protocols:

- leftmargin=* **By expert count:** $M = 1$ (ZK, commitments), $M = 2$ (two-party MPC), $M \geq 3$ (multi-party MPC, BFT, blockchain).

leftmargin=* **By gatekeeper exposure:** hidden (ZK, commitments), distributed (MPC, threshold), public (blockchain transactions).

leftmargin=* **By adversarial model:** semi-honest (MPC with $t < M/2$), malicious (BFT with $t < M/3$), rational (blockchain with majority assumption).

leftmargin=* **By audit frequency:** one-shot (ZK, commitments), interactive (MPC), repeated (BFT rounds, blockchain blocks).

This taxonomy reveals gaps: there are few protocols for $M > 1$ with fully hidden gatekeepers (distributed ZK), and few protocols for $M = 1$ with distributed gatekeepers (single-expert secret-shared audits). These represent promising directions for new cryptographic constructions.

Remark 10.2. The correspondence established in this paper, together with the SCX mappings for black hole thermodynamics [?] and quantum gravity [?], suggests a **unified audit theory spanning physics and cryptography**. Black holes seal information behind event horizons; cryptographic commitments seal secrets behind computational hardness; zero-knowledge proofs verify claims without revealing evidence. All are manifestations of the same information-theoretic structure: the Cercis bound on concealable information behind an audit boundary.

Appendix A: Summary of Theorems and Definitions

Table 2: Summary of all theorems, propositions, and definitions in this paper.

Label	Content
Definition ??	Zero-knowledge proof system
Definition ??	ZK proof as $(M = 1, g\text{-hidden})$ audit
Theorem ??	ZK completeness = SCX audit completeness
Theorem ??	ZK soundness = SCX audit soundness
Proposition ??	Zero-knowledge = gatekeeper secrecy
Theorem ??	Knowledge soundness = audit extractability
Proposition ??	Succinctness = audit efficiency
Definition ??	Secure multi-party computation
Definition ??	MPC as distributed audit
Proposition ??	MPC = distributed audit with secret-shared g
Definition ??	Byzantine consensus
Theorem ??	BFT = audit with f adversarial experts
Theorem ??	$f < M/3$ = audit collapse prevention
Definition ??	Nakamoto consensus
Theorem ??	Nakamoto consensus = $M > 1$ repeated audit
Proposition ??	Longest chain = audit majority
Theorem ??	Fork resolution = audit consensus
Definition ??	Verifiable random function
Definition ??	VRF as unbiased auditor selection
Theorem ??	VRF = unbiased auditor selection
Definition ??	Commitment scheme
Definition ??	Commitment as gatekeeper sealing
Proposition ??	Binding = gatekeeper sealing integrity
Proposition ??	Hiding = gatekeeper secrecy
Theorem ??	Pedersen = perfect audit binding
Definition ??	Threshold signature scheme
Definition ??	Threshold as audit quorum
Theorem ??	$t\text{-of-}n$ = audit quorum
Table ??	Complete 32-entry correspondence table
Definition ??	Universal audit correspondence
Theorem ??	Universal correspondence theorem

Appendix B: Numerical Verification

The Python script `verify_cryptography.py` accompanying this paper provides numerical verification of the key claims:

leftmargin=*, label=(1) **ZK completeness and soundness:** Simulated prover-verifier interactions verify that honest provers are accepted with high probability and dishonest provers are rejected.

- leftmbrgdn=*, llabel=(2) **BFT consensus under adversarial experts:** Simulated byzantine consensus with varying f/M ratios verifies the $f < M/3$ threshold.
- leftmrcgcn=*, llabel=(3) **MPC secret sharing as distributed audit:** Shamir secret sharing verifies that t shares reconstruct the secret while $t - 1$ shares reveal nothing.
- leftmdrgdn=*, llabel=(4) **VRF unbiased selection:** Simulated VRF-based auditor selection verifies uniform distribution and unpredictability.
- leftmergen=*, llabel=(5) **Threshold cryptography quorum:** Threshold signature verification confirms that t -of- n quorums function correctly.
- leftmfrgfn=*, llabel=(6) **Commitment binding and hiding:** Hash-based and Pedersen commitment schemes verify binding integrity and hiding secrecy.
- leftmgrggn=*, llabel=(7) **Blockchain fork resolution:** Simulated blockchain forks verify that the audit consensus converges to the longest chain.

All numerical claims can be reproduced by running:

```
python verify_cryptography.py
```

with `numpy` and `scipy` installed. Expected runtime: ~ 30 seconds.

References

- [1] SCX, *State-Conditioned Expertise: A Complete Theory of Label Noise Detection via Multi-Expert Consistency*, SCX Technical Report, 2026.
- [2] SCX, *Yajie: Elegant Purification of Label Noise via the Cercis Score*, SCX Technical Report, 2026.
- [3] SCX, *Spring: A Self-Evolving Gatekeeper with Provable Convergence*, SCX Technical Report, 2026.
- [4] SCX, *Black Hole Thermodynamics Meets SCX Audit: The No-Hair Theorem as Gauge Equivalence*, SCX Technical Report, 2026.
- [5] SCX, *Quantum Gravity Audit Equivalence*, SCX Technical Report, 2026.
- [6] S. Goldwasser, S. Micali, and C. Rackoff, *The knowledge complexity of interactive proof systems*, SIAM J. Comput. 18 (1989), 186–208.
- [7] J. Groth, *On the size of pairing-based non-interactive arguments*, Eurocrypt 2016, LNCS 9666, 305–326.
- [8] A. C. Yao, *Protocols for secure computations*, FOCS 1982, 160–164.
- [9] O. Goldreich, S. Micali, and A. Wigderson, *How to play any mental game*, STOC 1987, 218–229.
- [10] L. Lamport, R. Shostak, and M. Pease, *The Byzantine generals problem*, ACM Trans. Program. Lang. Syst. 4 (1982), 382–401.

- [11] S. Nakamoto, *Bitcoin: A peer-to-peer electronic cash system*, 2008.
- [12] M. Castro and B. Liskov, *Practical Byzantine fault tolerance*, OSDI 1999, 173–186.
- [13] G. Bracha, *Asynchronous Byzantine agreement protocols*, Inf. Comput. 75 (1987), 130–143.
- [14] D. Dolev and H. R. Strong, *Polynomial algorithms for multiple processor agreement*, STOC 1982, 401–407.
- [15] J. Garay, A. Kiayias, and N. Leonardos, *The Bitcoin backbone protocol: Analysis and applications*, Eurocrypt 2015, LNCS 9057, 281–310.
- [16] S. Micali, M. Rabin, and S. Vadhan, *Verifiable random functions*, FOCS 1999, 120–130.
- [17] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich, *Algorand: Scaling Byzantine agreements for cryptocurrencies*, SOSP 2017, 51–68.
- [18] M. Blum, *Coin flipping by telephone: A protocol for solving impossible problems*, IEEE Spring COMPCON 1982, 133–137.
- [19] T. P. Pedersen, *Non-interactive and information-theoretic secure verifiable secret sharing*, Crypto 1991, LNCS 576, 129–140.
- [20] Y. Desmedt and Y. Frankel, *Threshold cryptosystems*, Crypto 1989, LNCS 435, 307–315.
- [21] A. Shamir, *How to share a secret*, Commun. ACM 22 (1979), 612–613.
- [22] R. Cleve, *Limits on the security of coin flips when half the processors are faulty*, STOC 1986, 364–369.
- [23] Y. Lindell and B. Pinkas, *An efficient protocol for secure two-party computation in the presence of malicious adversaries*, Eurocrypt 2007, LNCS 4515, 52–78.
- [24] J. Watrous, *Zero-knowledge against quantum attacks*, STOC 2006, 296–305.